

Biomedical RAG

Config-driven Retrieval-Augmented Generation stack for biomedical and technical corpora, featuring hybrid retrieval, optional reranking, and multi-provider LLM orchestration.

Biomedical RAG 技术栈说明

1. 项目定位与总体思路

- **目标：**构建一个可配置、便于实验的 Retrieval-Augmented Generation (RAG) 管线，服务于生物信息/技术文档等 Markdown 形态的材料。
- **特性：**
 - Markdown 级别智能切分，保证标题、代码块等语义不被破坏。
 - 稠密向量检索 (Chroma) + 稀疏 BM25 的混合检索，支持 Weighted 与 RRF 两种融合策略。
 - FlagEmbedding BGE 模型驱动的可选 reranker，用于在生成前提升候选段落质量。
 - LLM 调度层可在 Kimi、OpenAI 兼容接口、DashScope 等多提供方之间切换。
 - 评估工具链 (notebook + HTML 可视化 + 离线数据集) 帮助验证检索质量和端到端回复。

2. 分层架构

2.1 配置层 (`config.py`, `settings/`)

- `config.py` 负责读取 `config.yaml`，并按顺序合并 `settings/base.yaml` 与 `settings/${env}.yaml`。
- 允许通过 `.env` 注入 API Key、模型端点等密钥信息，最终暴露 `CONFIG`、`ENV` 给各模块使用。
- 配置中可调：切分参数、retriever 默认 top_k、融合策略、reranker 开关、可用 LLM 列表、日志级别等。

2.2 数据切分层 (`rag/chunking/markdown_splitter.py`)

- 使用自定义 `MarkdownSplitter`，结合 `chunk_size` / `chunk_overlap` / 代码块保护策略，将 Markdown 文档拆分为结构化片段。

- 切分结果保留 `source`、`chunk_index`、`chunk_type` 等 metadata，后续用于检索去重与展示。

2.3 检索层 (`rag/retriever/`)

1. `DenseRetriever` (`dense.py`)

- 依赖 `EmbeddingProvider` 调用配置的向量模型。
- 使用 Chroma 向量存储 (`persist_dir` 默认 `data/chroma`)，支持增量/重建。

2. `BM25Retriever` (`bm25.py`)

- 采用 `rank-bm25` 进行稀疏检索。
- 通过 `BM25QueryTranslator` (LLM 驱动) 生成英文/中文关键词以提升召回。

3. `HybridRetriever` (`hybrid.py`)

- 聚合 dense 与 BM25 结果。
- 提供 Weighted 与 Reciprocal Rank Fusion (RRF) 两套融合算法。
- 负责去重 (按 `source` + `chunk index`) 和 `score` 标准化。

2.4 Reranker 层 (`rag/reranker/`)

- 默认实现 `BGEReranker` (FlagEmbedding + BAAI/bge-reranker-v2-m3)。
- 在 `rag.rag.RAGSystem` 中按需启用，支持配置 `batch size`、`FP16`、`device`。
- Rerank 后更新 metadata (`retriever_score`、`reranker_score`) 以便可视化与诊断。

2.5 LLM 生成层 (`rag/llm_client.py`, `rag/rag.py`)

- `LLMClient` 根据配置选择可用 LLM (OpenAI 兼容 / DashScope / Kimi 等)。
- `RAGSystem.generate_answer` 将检索结果 + prompt 模板 (来自 `prompts/`) 输入指定模型，返回最终回答。

2.6 评估与可视化 (`eval/`)

- `eval/end2end_eval.ipynb`：批量跑问答评测，生成 Markdown 报告。
- `eval/visualize_retrieval.py`：命令行脚本，展示 BM25 / Vector / Hybrid / Reranker 全流程，输出交互式 HTML。
- `eval/dataset/`：预制难例集、综合问题集，可直接驱动评估脚本。

3. 关键依赖与技术选型

组件	说明
Python 3.10+	项目主语言，开发时使用 3.13
LangChain Core	提供 prompt 模板、LLM 客户端封装等基础设施

组件	说明
ChromaDB	稠密向量数据库，持久化路径默认 <code>data/chroma</code>
Rank-BM25	经典 BM25 实现，用于稀疏检索
jieba	中文分词器，辅助 BM25 tokenization
FlagEmbedding + PyTorch	支撑 BGE reranker，GPU/FP16 优化
openai / DashScope / Moonshot SDK	访问不同 LLM 提供方
PyYAML	解析配置
pytest	核心模块的单元测试框架

4. 数据与缓存

- 文档来源：默认示例位于 `data/docs/Package_Documentation_md/...`，可替换为自有 Markdown。
- 缓存落地：
 - `data/chroma/` 下存放向量库、documents_cache JSON 等中间结果。
 - `logs/app.log` 记录运行日志（可在 `settings/*.yaml` 中配置）。
- 维护策略：
 - 运行 `rag.ensure_indexes()` 自动检查缓存是否存在/过期，不需要手动删除。
 - `rag.build_from_markdown_dir()` 强制重建，适合初次导入或大规模更新。

5. 配置与运行流程

1. 准备环境：`pip install -r requirements.txt`；创建 `.env` 写入 API Key。
2. 构建索引：

```
from rag.rag import RAGSystem
rag = RAGSystem()
rag.ensure_indexes()
```

3. 问答生成：

```
answer = rag.generate_answer("示例问题")
print(answer["answer"])
```

4. 评估/可视化：运行 `python eval/visualize_retrieval.py --query "..."`

所有参数（top_k、融合策略、rerank_k 等）均可通过 CLI 或 Python 调用时覆盖默认值。

6. 代码结构速览

目录/文件	用途
rag/	核心库（切分、嵌入、检索、rerank、LLM 调度）
eval/	评估脚本、notebook、HTML 报告
prompts/	查询翻译、生成提示模板
settings/	base/dev/prod 配置
tests/	Pytest 用例（splitter、retriever、vector store）
run_rag_example.py	最小可运行示例
README.md	高层使用指南
tech_stack.md	当前文档，聚焦技术栈介绍

7. 扩展方向与注意事项

- **多语种/多域**：可在 `BM25QueryTranslator` 中扩展更多查询改写模版，或针对不同语料启用特定 tokenizer。
- **自定义 reranker**：实现新的 `BaseReranker`，在 `settings/base.yaml` 中切换 `type` 即可。
- **向量库替换**：`EmbeddingProvider` 与 `HybridRetriever` 采用 LangChain 抽象，可扩展为其他 vector store。
- **生产部署**：结合 `config.prod.yaml` 配置日志、模型、API 限流；建议监控 `logs/` 与 `data/chroma` 占用。

Biomedical RAG Evaluation

1. Chunking

场景：本项目的知识库均为半结构化文本 Markdown 格式，采用 `langchain` 的 `MarkdownHeaderTextSplitter` 和 `RecursiveCharacterTextSplitter` 来分割文档。

LangChain

有些文档具有固有的结构，例如 HTML、Markdown 或 JSON 文件。根据文档结构进行

拆分会很有帮助，因为结构通常会将语义相关的文本自然地分组。基于结构的拆分的主要优势：

1. 保留文档的逻辑组织结构
2. 保持每个数据块内的上下文
3. 对于检索或摘要等下游任务来说可能更有效

遇到的问题：

1. 发现最初使用 `langchain` 的 `MarkdownHeaderTextSplitter` 无法正确识别 **Setext-style** 的 Markdown 标题。这个问题是通过可视化 Chunks 发现的。
 - 解决方案也很简单，使用正则表达式替换为 **ATX-style** 的 Markdown 表达式。
2. 发现知识库中 Markdown 包含代码示例，存在解释性的代码示例，和完整的大段的代码或代码输出。如果简单地按照正则表达式，对使用了代码块的部分进行提取，则会得到很多细碎的代码片段（只有一两行）。这个问题也是通过可视化 Chunks 发现的。
 - 解决方案是设定提取代码段的最小行数，设定 `min_lines_to_extract=10`。这样小于 10 行的代码会留在 Text Block 中，保留了上下文的语义信息。
 - 从 `HybridRetriever.build docs=6543` 到 `HybridRetriever.build docs=4010` 减少了 2500 个代码片段。

2. Embedding

场景：本项目采用了 Sparse Search 和 Dense Vector Search 相结合的检索策略。其中 Dense Vector Search 需要计算 Dense Embedding Vector。文档大部分为英文文档，但是 Query 经常是中文。所以需要选用 双语/多语言 Embedding 模型。

[mloasisblog](#)

Dense Embedding 能够捕捉文本的语义相似性，即使查询和文档使用了不同的词语，只要它们在含义上相近，Embedding 仍然可以将它们识别出来。

Dense Embedding 的原理是使用神经网络模型，如 Word2Vec、GloVe 和 Transformer 架构的模型（例如 Sentence-BERT 和 BGE）等，来学习文本的连续向量表示。这些模型通过训练学习了大量的文本数据，将每个文本 token 映射到一个低维的实数向量，向量中的每个维度都编码了文本的某些语义特征。

如何选择 Embedding 模型？

1. 训练 embedding 模型的成本极高，训练数据要几亿级 token，数百万 query-document pairs。
2. OpenAI、阿里云 embedding 等模型都是行业级 SOTA，再重新进行训练没有意义。
3. RAG 系统瓶颈并不在 embedding 模型上，而是在 Query Expansion、Chunking、Hybrid BM25 调参、RRF Rank Fusion、Reranker top-k、Metadata boost、Domain 关键词、

Query normalization 等。

4. 测试了 OpenAI `text-embedding-3-large` 和 阿里云百炼 `text-embedding-v4` 发现 `text-embedding-v4` 对于中英文的同义相似度更高，所以选择了这个模型。

3. Retrieval

场景：采用了在 RAG 场景下经常使用的 BM25 + Dense Vector Search。Fusion 方法采用了 RRF (Reciprocal Rank Fusion)。

| TODO: 补充 BM25 和 RRF 相关知识

遇到的问题

1. 在 End-to-End 评估中，发现 Contextual Relevancy 很低，然后仔细查看了一下检索各部分的结果，发现 BM25 无法跨语言检索。所以在执行 BM25 前，使用 LLM 重写 Query 为英文，这样 BM25 执行两次，一次为中文检索，一次为英文检索，最后输出 top-k。
2. 最开始只使用 Weighted Fusion，发现一些包含关键词的简短片段往往在 top-k 的前几名。所以使用 RRF，RRF 具有天生“鲁棒性”，不需要调权重。（weight tuning 非常难）。通过可视化 Retrieval 结果发现 RRF 的结果确实比 Weighted 要好。

4. Rerank

场景：dense/BM25 检索理解不到位导致排名错误，他们都是粗召回、粗排序。容易把不相关的 chunk 排到前面。采用了开源的 `BAAI/bge-reranker-v2-m3` reranker。由于 Query/Document 的双语属性，所以采用多语言模型。

选用 `BAAI/bge-reranker-v2-m3` 原因：

- Reranker 类第一梯队
- 参数量小（~100M 左右）
- 经济考量
- 其他参考（未评估）：
 - `Jina-reranker-v2-base-multilingual`（开源）
 - `Cohere Rerank v3`（付费）

遇到的问题

1. 在 eval 后，发现了一个 query 的 top-k 召回结果中没有 golden chunk。观察后发现找回到了 golden chunk 的上一个 chunk。这是由于 chunking 受到了 `chunk_size` 和 `chunk_overlap` 的限制，导致包含 golden text 的 chunk 的前面包含了语义无关的内容，被 vector 检索和 rerank 抛弃。
 - 解决方法：遍历 rerank 后得到的 top-k chunks，检查是否存在子 chunk，如果存在，则合并下一个最接近的子 chunk。

- *TODO：是否有更智能的方案，解决这个问题？比如用 LLM 识别哪些子 chunk 与 query 最相关等。*

5. Generation

System Prompt

你是一名专业的生物信息学与数据分析顾问。严格依据提供的资料回答问题；若资料不足以支撑结论，请明确说明无法回答。回答使用中文，同时保留必要的英文专业术语。当问题涉及操作方法、函数使用或数据分析流程时，必须直接提供代码示例，不要询问用户是否需要。

User Prompt

请根据以下检索资料回答用户问题，并遵循这些准则：

1. 回答必须建立在资料内容之上，避免主观推测。
2. 如资料包含代码或命令，请完整引用并保持原格式（包括代码块标记）。
3. 如果问题询问如何操作、如何使用函数或如何实现某个功能，必须直接提供代码示例，不要询问是否需要：
 - 优先使用资料中已有的代码示例；
 - 如果资料中有相关函数或方法的说明，基于这些说明构造可运行的代码示例；
 - 代码示例应包含必要的库导入、参数设置和完整调用；
 - 直接在回答中包含代码块，不要问“你需要我帮你写这个代码吗”之类的问题。
4. 语言保持专业、简洁，可使用分点阐述重点。
5. 如果资料不足以得出完整答案，请直接说明缺失信息。
6. 直接进入回答，不要使用“基于上下文”等开场白，也不要询问用户是否需要额外帮助。

【检索到的资料】

{context}

【用户问题】

{query}

请给出你的回答：

6. Evaluation

Evaluation 部分是整个 RAG 系统中最难的部分，RAG 的输出链路非常复杂，从 query → retrieval → reranking → generation，每一环都有误差，每一环的误差会相互放大。

所以 RAG 的评估本质上是一道“多维贝叶斯误差传播”问题，是整个链路里最难、最不确定、最主观的部分。

RAG 构建很容易：

- 找个 embedding
- 加 BM25

- 让 LLM 回答
 - 用 RRF + reranker
- 就能跑起来。

但评估则：

- 无 ground truth
- 多维度
- 不确定
- 难自动化
- error source 难定位
- trade-off 极多
- LLM-as-judge 本身带噪声

6.1 测试集

6.2 端到端测试

端到端测试使用 LLM-as-Judge，使用开源框架 DeepEval 提供的 RAG 端到端的 5 个评估指标（Answer Relevancy, Faithfulness, Contextual Precision, Contextual Recall, Contextual Relevancy）。

Answer Relevancy

答案相关性指标使用 LLM 作为评判标准，通过评估 LLM 应用的实际输出与所提供输入的相关性来衡量 RAG 管道生成器的质量。deepeval 的答案相关性指标是一个自解释的 LLM 评估，这意味着它会输出其指标得分的原因。

Faithfulness

忠实度指标使用 LLM 作为评判标准来衡量 RAG 管道生成器的质量，评估实际输出是否与检索上下文的内容完全一致。deepeval 的忠实度指标是一个自解释的 LLM 评估，这意味着它会输出指标得分的原因。

Contextual Precision

上下文精确度指标使用 LLM 作为评判标准来衡量 RAG 管道检索器的性能，具体方法是评估检索上下文中与给定输入相关的节点是否比不相关的节点排名更高。deepeval 的上下文精确度指标是一个自解释的 LLM-Eval，这意味着它会输出指标得分的原因。

Contextual Recall

上下文召回率指标使用 LLM 作为评判标准，通过评估检索上下文与预期输出的匹配程度来衡量 RAG 管道检索器的质量。deepeval 的上下文召回率指标是一个自解释的 LLM 评估，这意味着它会输出指标得分的原因。

Contextual Relevancy

上下文相关性指标使用 LLM 作为评判标准，通过评估检索上下文中呈现的信息与给定输入的整体相关性来衡量 RAG 管道检索器的质量。deepeval 的上下文相关性指标是一个自解释的 LLM-Eval，这意味着它会输出其指标得分的原因。

选择了 10 条测试数据，测试结果如下：

评估指标 (Metric)	平均分 (Average Score)	表现评级	核心含义
Faithfulness (忠实度)	0.9905	🌟 极好	生成的答案是否忠实于检索到的上下文（无幻觉）。
Contextual Precision (上下文精确度)	0.9167	✅ 优秀	相关文档在检索结果中的排序是否靠前。
Answer Relevancy (答案相关性)	0.9163	✅ 优秀	生成的答案是否直接回答了用户的问题。
Contextual Recall (上下文召回率)	0.9000	✅ 优秀	检索到的文档是否包含了回答问题所需的全部信息。
Contextual Relevancy (上下文相关性)	0.5824	⚠️ 较差	检索到的文档内容中有多少比例是真正与问题相关的（信噪比）。